

Modül 02: Değişkenler ve Tipler

Değişkenin ne olduğunu biliyorsunuz. Java'dan ihtiyacınız olan şey daha dar: hangi tipler var, bunların hangisi nesne hangisi değil, ve bu ikisi arasındaki sınır sizi nerede yanıltacak. O sınır bu modülün tamamı.

Neler Öğreneceksiniz

- İlkel tipler ile referanslar arasındaki ayrım ve Java'nın bunu neden hâlâ koruduğu
- İnsanları şaşırtan ama hiçbir hata olmayan beş sayısal davranış
- Otomatik kutulama ve ısırdığı iki yer
- `==` operatörünün nesneler hakkında neden yalan söylediği ve yerine ne kullanılacağı
- Değişmez nesneler olarak String'ler ve metin blokları
- `var`, ne yaptığı ve nerede yardımı kesildiği

Başka Bir Dilden Geliyorsanız

Python ve JavaScript'te sayılar dahil her şey nesnedir. Java iki ayrı dünyayı korur ve dikişi hissedeceksiniz.

	Python / JS	Java
<code>3</code>	bir nesne	ilkel tip, 32 bit, tahsis yok
Tanımlama	<code>x = 3</code>	<code>int x = 3;</code> ya da <code>var x = 3;</code>
Sonradan tip değiştirme	sorun değil	imkânsız, tip derleme zamanında sabitlenir
Tam sayı taşması	sayıyı büyütür	sessizce başa sarar
<code>"a" == "a"</code>	içeriği karşılaştırır	kimliği karşılaştırır, <code>.equals</code> kullanın
Doğruluk kavramı	<code>if (items)</code> çalışır	yalnızca <code>boolean</code> kabul edilir

Son satır bir not hak ediyor. Java'da doğruluk kavramı yok. `if (someString)` derlenmez. `if (!someString.isEmpty())` yazarsınız, ve bu uzunluk bilinçlidir: hangi değerlerin yanlış sayıldığına dair ezberlenecek bir kural yoktur.

Ders

İki tür değer

Sekiz ilkel tip var ve bu liste sonsuza dek kapalı:

```
boolean    int    long    double
char       byte   short   float
```

İlkel tipli bir değişken değerin kendisini tutar. Java'daki diğer her şey bir **referanstır**: değişken, başka bir yerde yaşayan bir nesnenin adresini tutar.

Fark, bir şeyi kopyalayana kadar görünmez. `TwoKindsOfValue.java` dosyasına bakın:

```
int[] original = {1, 2, 3};
int[] alias = original; // copies the reference, not the array
alias[0] = 999;
// original[0] is now 999
```

İlkel tipi kopyalamak değeri kopyalar. Referansı kopyalamak etiketi kopyalar ve iki etiket de aynı nesneyi gösterir. Modül 04, metod argümanlarına baktığımızda buna geri dönüyor.

Pratikte `int`, `long`, `double`, `boolean` ve `char` kullanacaksınız. Diğer üçü çoğunlukla karşılaşmanız pek olası olmayan bellek kısıtlı durumlar için var.

Sayılar sizi şaşırtacak

`NumbersWillSurpriseYou.java` dosyasını çalıştırın. Gerçek çıktı:

```
7 / 2 = 3
7 / 2.0 = 3.5
Integer.MAX_VALUE + 1 = -2147483648
Math.addExact says: integer overflow
0.1 + 0.2 = 0.30000000000000004
7 % -2 = 1
-7 % 2 = -1
Math.floorMod(-7, 2) = 1
'A' + 1 = 66
```

Orada beş şey oluyor:

1. **Tam sayı bölmesi kalanı atar.** `7 / 2` sonucu `3`. Bir işleneni `double` yapın, `3.5` alırsınız. İki işlenen de değişkenken bunu göremezsiniz, tehlikeli olan yanı bu.
2. **Taşma sessizce başa sarar.** `int` 32 bittir ve en büyüğüne bir eklemek istisna olmadan en küçüğe döner. İstediginizde `Math.addExact` şikâyet eder.
3. **`double` çoğu ondalığı tam tutamaz.** `0.1` ikilik tabanda devirli bir kesirdir. Parayı asla `double` içinde saklamayın. Kuruş cinsinden bir `long` ya da `BigDecimal` kullanın.
4. **% sol işlenenin işaretini korur.** `-7 % 2` sonucu `-1`, `1` değil. Matematiksel modülo istiyorsanız o `Math.floorMod`.
5. **`char` bir sayıdır.** `'A' + 1` sonucu `66`, çünkü `+` `char`'ı `int`'e yükseltti. `B` almak için `(char)` ile geri dönüştürün.

Otomatik kutulama ve iki ısırığı

Her ilkel tipin bir nesne ikizi var: `int` / `Integer`, `double` / `Double` ve devamı. Java bunlar arasında otomatik dönüşüm yapar. Buna otomatik kutulama denir ve şu iki âna kadar gerçekten kullanışlıdır.

Birinci ısırık, kutulanmış değerlerde `==`. `TheAutoboxingTrap.java` dosyasından:

```
127 == 127 : true
128 == 128 : false
```

Aynı kod, aynı türden değer, zıt cevap. Java `-128..127` aralığı için `Integer` nesnelerini ön belleğe alır, dolayısıyla bu aralıkta `==` bir nesneyi kendisiyle karşılaştırır. Aralığın dışında iki nesne alırsınız ve `==` doğru biçimde farklı nesneler olduklarını bildirir.

Bozuk olan bir şey yok. Nesnelerde `==` her zaman "aynı nesne" anlamına geliyordu, ön bellek bunu sizden gizliyordu.

Kural: ilkel tiplerde `==`, nesnelerde `.equals()`.

İkinci ısırık, null kutudan çıkarma. `NullUnboxing.java` bilerek çöker:

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "java.lang.Integer.intValue()" because the return value of
  "java.util.Map.get(Object)" is null
```

`Map.get` olmayan bir anahtar için `null` döndürür ve bunu bir `int` değişkene atamak Java'yı hiçliğin üzerinde `intValue()` çağırma zorlar. Mesajın ne kadar belirgin olduğuna dikkat edin: `null` döndüren çağrıyı ve onun üzerinde başarısız olan çağrıyı isimlendiriyor. Modern Java NPE'leri hata ayıklamanın çoğunu sizin için yapar.

String'ler

String'ler nesnedir ve **değişmezdir**. Hiçbir metot, üzerinde çağırdığınız string'i değiştirmez.

`StringsAndTextBlocks.java` dosyasından:

```
String name = "ada";
name.toUpperCase();           // computes "ADA" and throws it away
name = name.toUpperCase();    // you have to keep the result
```

İlk satır insanların gerçekten yaptığı bir hatadır ve hiçbir şey sizi uyarmaz.

`==` tuzağı burada da geçerli, `Integer` ile aynı sebepten:

```
literal == literal : true      // derleyici aynı sabitleri havuzlar
literal == new     : false     // new String() ayrı bir nesne zorlar
equals           : true
```

Hemen edinilmeye değer bir alışkanlık: sabiti sola koyun.

```
"java".equals(maybeNull)    // false
maybeNull.equals("java")   // NullPointerException
```

Metin blokları kaçış karakteri olmadan çok satırlı string verir:

```
String json = """
{
    "name": "Ada"
}""";
```

Java ortak baştaki girintiyi, en az girintili satırdan ölçerek siler, böylece blok kodunuzla hizalanır ama o boşluk değere karışmaz.

var

`var`, derleyiciden tipi sağ taraftan çıkarmasını ister. Bu kısmı dikkatle okuyun, çünkü insanların yanlış anladığı nokta burası:

`var` dinamik tiplere değildir. Değişkenin hâlâ tam olarak bir tipi vardır, sonsuza dek sabitlenmiş, derleme zamanında denetlenir. Siz sadece yazmadınız.

```
var x = "hello";
x = 42;    // error: incompatible types: int cannot be converted to String
```

Tip uzun ve zaten görünür olduğunda hakkını verir:

```
var scores = new HashMap<String, List<Integer>>();
```

Sağ taraf ne aldığınızı söylemiyorsa zarar verir:

```
var result = service.process(input);    // now go look up process()
```

Yapamadığı üç şey, hepsi derleme hatası:

```
var a;                // nothing to infer from
var b = null;         // null belongs to every reference type
var c = () -> 1;      // a lambda needs a target type
```

`VarCannotInferNull.java` ikincisini gösteriyor. Ayrıca yalnızca yerel değişkenler içindir. Alanlar, parametreler ve dönüş tipleri hâlâ yazılmalıdır, bu da API'leri okunabilir tutar.

Çalıştırın

```
java modules/02-variables-and-types/examples/TwoKindsOfValue.java
java modules/02-variables-and-types/examples/NumbersWillSurpriseYou.java
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java
java modules/02-variables-and-types/examples/StringsAndTextBlocks.java
java modules/02-variables-and-types/examples/VarAndInference.java

# Bu ikisi bilerek başarısız olur. Ne dediklerini okuyun.
java modules/02-variables-and-types/examples/NullUnboxing.java
java modules/02-variables-and-types/examples/VarCannotInferNull.java

./scripts/verify-examples.sh modules/02-variables-and-types
```

Sık Yapılan Hatalar

String karşılaştırmak için `==` kullanmak. Testinizde kısa sabitlerle çalışır, üretimde çalışma zamanında oluşturulan string'lerle başarısız olur. Yeni başlayanların yazdığı en yaygın Java hatası budur. `.equals` kullanın.

Parayı `double` içinde saklamak. `0.1 + 0.2` sonucu `0.3` değildir. Kuruş cinsinden `long` ya da `BigDecimal` kullanın. Muhasebe sistemleri yanlış yazılmıştır ve tutarsızlık aylar sonra ortaya çıkar.

String metotlarının yeni bir String döndürdüğünü unutmak. Tek başına bir satırda `name.strip();` hiçbir şey yapmaz.

`Map.get` sonucunun bir `int` içine kutudan çıkmasına izin vermek. Eksik her anahtar `NullPointerException` olur. `getOrDefault` kullanın ya da `Integer` olarak tutup kontrol edin.

Sıcak döngü içinde kutulama.

```
Long sum = 0L; // a million allocations
for (long i = 0; i < 1_000_000; i++) sum += i;
```

`Long` yerine `long` yazmak her tahsisi kaldırır. İki sürüm neredeyse aynı görünür, bunu bilmeye değmesinin sebebi de tam olarak bu.

`long big = 9000000000L;` yazmak, `L` olmadan. Sabit önce `int` olarak ayrıştırılır, sığmaz, dolayısıyla derlenmez.

Özet Çıkarımlar

- **İki dünya.** İlkel tipler değer tutar, referanslar adres tutar. Referansı kopyalamak nesneyi değil etiketi kopyalar.
- `==` "aynı nesne mi" diye sorar, `.equals` "aynı içerik mi" diye. İlkel olmayan her şeyde istediğiniz `.equals`.

- **Integer önbellegi** `==` işaretini 128'in altında doğru gösterir. Onu tehlikeli yapan sınır değil, bu.
- **Sayılar sabit genişliktedir.** Tam sayı bölmesi keser, taşma başa sarar ve `double` çoğu ondalığı tam tutamaz.
- **String'ler asla değişmez.** Her metot yenisini döndürür, sonucu saklayın.
- `var` çıkarımdır, dinamik tipleme değil. Tip hâlâ sabit ve hâlâ denetleniyor.
- **Java'da doğruluk kavramı yoktur.** `if` içine yalnızca `boolean` girer, ve bu bir özelliktir.

Ödev

[homework/README.md](#)

Para işlemeyi doğru yapan küçük bir fiş hesaplayıcı kurun, sonra bu modülün iki tuzağını bilerek yeniden üretin. Referans çözüm [solutions/02-variables-and-types/](#) altında.